

# Version 1 Method Design Doc

## Textual-Feedback DPO for Structured Self-Reflective Small Language Models

*First controlled prototype: Search-QA + Math*

**Core idea**

Train Qwen3-4B to produce structured trajectories with <plan>, <think>, <tool>, <reflect>, and <final>, using one simple DPO objective: corrected rollout > original student rollout. The teacher gives textual feedback only. Reflection must include verification before the final answer.

| Component        | V1 choice                                                                                  |
|------------------|--------------------------------------------------------------------------------------------|
| Student          | Qwen3-4B-Instruct                                                                          |
| Teacher          | Frozen stronger model, such as Qwen3-32B or another strong model                           |
| Domains          | Search-QA and Math                                                                         |
| Training         | Offline DPO plus small format SFT                                                          |
| Tools            | Controlled search for QA; optional Python verification for math                            |
| Excluded from V1 | OpenHands/SWE, trainable teacher, memory tools, RL, focus weights, multiple DPO objectives |

## Table of Contents

- 1. Scope and research question
- 2. Benchmarks and datasets
- 3. Policy and trajectory format
- 4. Tool policy
- 5. Reflection and verification policy
- 6. Teacher role and feedback policy
- 7. DPO data construction
- 8. Loss function
- 9. End-to-end algorithm and pseudocode
- 10. Prompt templates
- 11. Evaluation and ablations
- 12. Implementation order
- 13. Hyperparameters
- 14. Later extensions
- 15. References

## 1. Scope and research question

V1 is a controlled research prototype. It is intentionally simpler than the final agentic vision. The purpose is to test the main mechanism before adding memory, SWE/OpenHands tasks, online RL, or teacher training.

### 1.1 One-line method

```
student original rollout
  -> teacher textual feedback
  -> corrected rollout
  -> DPO: corrected rollout > original rollout
```

### 1.2 Main research question

Can a 4B student model learn a better reasoning policy when teacher feedback repairs its own failed structured rollout, and DPO trains the student to prefer the repaired rollout over the original failure?

### 1.3 What V1 includes

- Student model: Qwen3-4B-Instruct.
- Teacher model: frozen stronger model.
- Domains: Search-QA and Math.
- Structure: <plan>, <think>, <tool>, <reflect>, <final>.
- Mandatory verification inside <reflect>.
- Offline DPO with one preference objective.
- Small format SFT to stabilize the structured output style.
- Teacher-written corrected rollouts for the first version.

### 1.4 What V1 excludes

- OpenHands, Qwen Code, SWE-agent, or software-engineering repo tasks.
- Trainable teacher or feedbacker.
- Memory tools or cross-episode memory.
- Multiple DPO objectives.
- Numeric focus weights.
- GRPO, PPO, online RL, or online teacher adaptation.
- Long autonomous tool loops.

## 2. Benchmarks and datasets

V1 uses two controlled domains. Search-QA tests tool/evidence use and multi-hop planning. Math tests reasoning and verification.

### 2.1 Search-QA

| Role                     | Dataset           | Why it is useful                                                                                     |
|--------------------------|-------------------|------------------------------------------------------------------------------------------------------|
| First train              | HotpotQA          | Multi-hop questions with supporting facts; useful for planning, evidence collection, and reflection. |
| Add after pipeline works | Natural Questions | Real Google-style questions with Wikipedia answers; more realistic search-style distribution.        |
| Evaluate                 | HotpotQA held-out | In-domain multi-hop test.                                                                            |
| Evaluate                 | NQ held-out       | In-domain realistic question test.                                                                   |

|                |                     |                                                      |
|----------------|---------------------|------------------------------------------------------|
|                |                     |                                                      |
| Generalization | 2WikiMultiHopQA     | Multi-hop QA with reasoning-path evidence.           |
| Generalization | MuSiQue             | Designed to reduce shortcut-solving in multi-hop QA. |
| Later          | TriviaQA, Bamboogle | Broader QA and hard out-of-domain checks.            |

## 2.2 Math

| Role                     | Dataset             | Why it is useful                                                |
|--------------------------|---------------------|-----------------------------------------------------------------|
| First train              | GSM8K               | Multi-step grade-school word problems with clean exact answers. |
| Add after pipeline works | MATH easy subset    | Harder symbolic/contest-style reasoning.                        |
| Evaluate                 | GSM8K held-out/test | In-domain math reasoning.                                       |
| Evaluate                 | MATH-500            | Compact harder math evaluation.                                 |
| Later                    | SVAMP               | Variation-focused word-problem generalization.                  |

## 2.3 Dataset sizes

| Stage               | Data size                                       | Goal                                                                                 |
|---------------------|-------------------------------------------------|--------------------------------------------------------------------------------------|
| Stage 0: smoke test | 100 HotpotQA + 100 GSM8K                        | Check parser, tool execution, teacher correction quality, and DPO data construction. |
| Stage 1: dry run    | 500 HotpotQA + 500 GSM8K, with 100 val each     | See if DPO moves accuracy or format behavior at all.                                 |
| Stage 2: real V1    | 2k HotpotQA + 2k GSM8K + 500 NQ + 500 MATH easy | Train the first serious mixed-domain adapter.                                        |

## 3. Policy and trajectory format

The student policy generates structured trajectories. The structure is not a rigid loop. A plan may create one branch, two branches, or several local reasoning/tool segments before reflection aggregates them.

```

<plan>
High-level meta-plan.
State the problem type.
State the route or routes.
State branch budget.
State tool needs.
State what must be verified before final answer.
</plan>

<think branch="A">
Local reasoning for branch A.
</think>

<tool branch="A">
Optional tool call and observation.
</tool>

```

```

<think branch="B">
Optional local reasoning for branch B.
</think>

<tool branch="B">
Optional tool call and observation.
</tool>

<reflect>
Branch comparison:
...

Evidence / derivation check:
...

Verification:
...

Decision:
continue / answer
</reflect>

<final>
Final answer only.
</final>

```

### 3.1 Meaning of each bracket

| Bracket   | Meaning                                                                                    | What it should not do                                |
|-----------|--------------------------------------------------------------------------------------------|------------------------------------------------------|
| <plan>    | Meta-level planning: what to do, what branches to try, what tool/evidence/check is needed. | It should not be a long detailed solution.           |
| <think>   | Local scratchpad reasoning for one branch/subgoal.                                         | It should not handle the entire global strategy.     |
| <tool>    | External evidence/computation: search result, Python check, or observation.                | It should not contain invented observations.         |
| <reflect> | Meta-level aggregation and verification before answer.                                     | It should not be decorative summary; it must verify. |
| <final>   | Final answer only.                                                                         | It should not include long reasoning.                |

### 3.2 Branching policy

- Search-QA: one branch for simple direct factual questions; two branches for likely multi-hop questions; maximum three branches in V1.
- Math: one branch for direct algebra/arithmetic; two branches when there is ambiguity or a high risk of calculation/setup error; maximum two branches in V1.
- The model should not branch just to look careful. Branches should test distinct routes or checks.

## 4. Tool policy

Tools are intentionally simple in V1. They are used to test agentic behavior without introducing a full software-agent harness.

## 4.1 Search-QA tool

```
search(query) -> top-k passages
```

V1 defaults:

- Use controlled retrieval, not live web.
- Start with BM25 over a Wikipedia/dataset corpus.
- top\_k = 3 initially; top\_k = 5 later.

The model may request search in a <tool> block, but the harness must insert the actual observation. The model should not fabricate search results.

## 4.2 Math tool

```
python(code) -> output
```

Allowed in V1:

- arithmetic checks
- substitution checks
- simple numeric verification

Not allowed in V1:

- asking Python to solve the whole problem automatically

The intended behavior is: reason first, then use Python for verification if useful.

## 5. Reflection and verification policy

<reflect> is the gate before <final>. It must include a verification section. This is one of the central design choices of V1.

```
<reflect>
Branch comparison:
...

Evidence / derivation check:
...

Verification:
...

Decision:
continue / answer
</reflect>
```

### 5.1 Search-QA verification checklist

1. Entity check: Did the trajectory identify the right subject or intermediate entity?
2. Relation check: Did it answer the relation asked by the question?
3. Evidence check: Is the answer supported by retrieved observations?
4. Type check: Does the answer type match the question, such as person, place, date, number, or object?
5. Decision check: Should the model continue searching or answer now?

### 5.2 Math verification checklist

6. Arithmetic/algebra check: Did the derivation follow valid steps?
7. Substitution check: Can the answer be plugged back into the original condition?
8. Constraint check: Does the answer satisfy sign, integer, range, unit, or context constraints?
9. Branch agreement: If multiple branches were used, do they agree?

## 6. Teacher role and feedback policy

The teacher is frozen in V1. It reviews the student rollout and writes textual feedback plus a corrected rollout. It does not output numeric focus weights.

### 6.1 Teacher input

```
problem x
gold answer
student rollout tau_0
tool observations
student final answer
correctness result
```

### 6.2 Teacher output

```
<feedback>
Textual feedback.
</feedback>

<corrected_rollout>
Corrected structured trajectory.
</corrected_rollout>
```

### 6.3 What the feedback should contain

| Feedback type          | Meaning                                        | Examples                                                                                                                        |
|------------------------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Computation allocation | What cognitive operation should be changed.    | Use two branches, not one; use less repeated search; reflect before final; verify the answer type.                              |
| Content correction     | What was actually wrong inside the trajectory. | Plan skipped the intermediate entity; query was too broad; reflection chose the weaker branch; arithmetic verification missing. |

#### Reveal policy

The feedback should usually avoid directly revealing the final answer. The corrected rollout may contain the answer because it is the chosen training target, but the feedback itself should teach the repair rather than spoon-feed.

## 7. DPO data construction

The DPO dataset contains preference pairs generated from the student and teacher.

```
For each example:
1. Student generates original rollout tau_0.
2. Evaluate tau_0.
3. Teacher writes feedback and corrected rollout tau_plus.
4. Evaluate tau_plus.
5. If tau_plus is better, store:
    prompt = original problem + format/tool instructions
    chosen = tau_plus
    rejected = tau_0
```

### 7.1 Important prompt choice

For the main DPO objective, use the original problem prompt as the DPO prompt. Do not include the teacher feedback in the prompt. This makes the student internalize the corrected behavior so inference does not require a teacher.

DPO prompt:  
 problem + format/tool instructions

Chosen:  
 corrected rollout

Rejected:  
 original student rollout

## 7.2 Pair filtering

- First run: keep only pairs where the original is wrong and the corrected rollout is correct.
- Require valid bracket format in the corrected rollout.
- Require verification inside <reflect>.
- Later: optionally keep pairs where both are correct but the corrected rollout has better verification or fewer unnecessary tool calls.

## 8. Loss function

V1 uses one DPO loss plus a small format SFT loss. Do not add multiple DPO losses in the first implementation.

### 8.1 DPO loss

```
For each pair (x, y_plus, y_minus):

L_DPO = -log sigma(beta * [
    log pi_theta(y_plus | x) - log pi_ref(y_plus | x)
  - log pi_theta(y_minus | x) + log pi_ref(y_minus | x)
])

Recommended starting beta: 0.1
```

### 8.2 Format SFT loss

```
L_FormatSFT = - sum_t log pi_theta(y_plus_t | x, y_plus_<t)
```

### 8.3 Final V1 objective

```
L_total = L_DPO + 0.1 * L_FormatSFT
```

DPO is appropriate here because the data is naturally comparative: the corrected rollout should be preferred over the original failure. It avoids the complexity of learning a separate reward model or running online RL during the first version.

## 9. End-to-end algorithm and pseudocode

### 9.1 Rollout generation

```
def generate_original_rollout(student, example, domain, tools):
    prompt = build_student_prompt(example, domain, tools)
    rollout = student.generate(
        prompt,
        max_tokens=2048,
        temperature=0.7,
    )

    rollout = execute_tools_if_needed(rollout, tools)
    result = evaluate_rollout(rollout, example, domain)
```

```

return {
    "prompt": prompt,
    "rollout": rollout,
    "result": result,
}

```

## 9.2 Tool execution

```

def execute_tools_if_needed(rollout, tools):
    # Parse <tool> blocks.
    # If search(query), call the retriever and insert top-k observations.
    # If python(code), run restricted Python and insert the output.
    # Return the completed rollout with real observations.
    return completed_rollout

```

## 9.3 Teacher correction

```

def get_teacher_correction(teacher, example, original_trace, domain):
    teacher_prompt = build_teacher_prompt(
        problem=example["question"],
        gold_answer=example["answer"],
        student_rollout=original_trace["rollout"],
        result=original_trace["result"],
        domain=domain,
    )

    teacher_output = teacher.generate(
        teacher_prompt,
        max_tokens=3000,
        temperature=0.2,
    )

    feedback = parse_feedback(teacher_output)
    corrected_rollout = parse_corrected_rollout(teacher_output)
    corrected_result = evaluate_rollout(corrected_rollout, example, domain)

    return {
        "feedback": feedback,
        "corrected_rollout": corrected_rollout,
        "corrected_result": corrected_result,
    }

```

## 9.4 Pair filtering

```

def is_useful_pair(original_result, corrected_result):
    if not corrected_result["format_valid"]:
        return False

    if not corrected_result["verification_present"]:
        return False

    # First V1 filtering rule: original wrong, corrected correct.
    if original_result["score"] < corrected_result["score"]:
        return True

    return False

```

## 9.5 DPO dataset builder

```

def build_dpo_dataset(examples, student, teacher, domain, tools):
    dpo_pairs = []
    format_sft = []

```

```

for example in examples:
    original = generate_original_rollout(student, example, domain, tools)
    correction = get_teacher_correction(teacher, example, original, domain)

    if is_useful_pair(original["result"], correction["corrected_result"]):
        dpo_pairs.append({
            "prompt": original["prompt"],
            "chosen": correction["corrected_rollout"],
            "rejected": original["rollout"],
            "domain": domain,
            "metadata": {
                "original_score": original["result"]["score"],
                "corrected_score": correction["corrected_result"]["score"],
                "feedback": correction["feedback"],
            },
        })

        format_sft.append({
            "prompt": original["prompt"],
            "completion": correction["corrected_rollout"],
            "domain": domain,
        })

return dpo_pairs, format_sft

```

## 9.6 Training loop

```

def train_v1(student_model, reference_model, dpo_pairs, format_sft):
    for batch in dataloader(dpo_pairs):
        dpo_loss = compute_dpo_loss(
            policy=student_model,
            reference=reference_model,
            prompt=batch["prompt"],
            chosen=batch["chosen"],
            rejected=batch["rejected"],
            beta=0.1,
        )

        sft_batch = sample(format_sft)
        sft_loss = compute_sft_loss(
            policy=student_model,
            prompt=sft_batch["prompt"],
            completion=sft_batch["completion"],
        )

        loss = dpo_loss + 0.1 * sft_loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

```

## 10. Prompt templates

### 10.1 Student prompt

You are solving a problem using a structured reasoning policy.

You must use this format:

<plan>

High-level meta-plan. Decide the route, number of branches, tools needed, and verification needed before final answer.

```

</plan>

<think branch="A">
Local reasoning for branch A.
</think>

<tool branch="A">
Optional tool call and observation.
</tool>

<reflect>
Branch comparison:
Evidence / derivation check:
Verification:
Decision:
</reflect>

<final>
Final answer only.
</final>

Rules:
1. Do not give <final> before <reflect>.
2. <reflect> must contain verification.
3. Use at most 3 branches for Search-QA and at most 2 branches for Math.
4. Use tools only when useful.
5. The final answer must be concise.

Problem:
{problem}

```

## 10.2 Search-QA addition

For Search-QA, verify:

- entity
- relation
- evidence support
- answer type

## 10.3 Math addition

For Math, verify using:

- arithmetic check
- substitution
- constraints
- sign/range/unit
- branch agreement if multiple branches are used

## 10.4 Teacher prompt

You are a teacher correcting a small language model's structured rollout.

The student must use:  
 <plan>, <think>, <tool>, <reflect>, <final>.

Your job:

1. Give textual feedback.
2. Explain what computation should change:
  - more/less planning, thinking, tool use, branching, reflection, verification.
3. Explain what content should change:
  - wrong route, wrong query, missing evidence, bad arithmetic, wrong final type, etc.
4. Produce a corrected rollout in the same format.
5. Do not add unnecessary branches.
6. Ensure <reflect> contains real verification.

7. Ensure `<final>` contains only the final answer.

```

Problem:
{problem}

Gold answer:
{gold_answer}

Student rollout:
{student_rollout}

Student result:
{result}

Return exactly:

<feedback>
...
</feedback>

<corrected_rollout>
...
</corrected_rollout>

```

## 11. Evaluation and ablations

### 11.1 Systems to evaluate

| Experiment | Description                                                        | Purpose                                          |
|------------|--------------------------------------------------------------------|--------------------------------------------------|
| E0         | Base Qwen3-4B normal prompt                                        | Raw baseline.                                    |
| E1         | Base Qwen3-4B bracket prompt, no training                          | Tests whether prompting alone helps.             |
| E2         | Format SFT only                                                    | Tests whether format imitation alone helps.      |
| E3         | DPO + 0.1 FormatSFT                                                | Main V1 method.                                  |
| E4         | DPO without mandatory verification in <code>&lt;reflect&gt;</code> | Tests whether verification actually contributes. |

### 11.2 Metrics

| Category  | Metrics                                                                                                                               |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------|
| Common    | Final answer accuracy, format validity, verification-present rate, verification-valid rate, average tokens, branch count, tool calls. |
| Search-QA | Exact Match, F1, answer type correctness, evidence support rate, search calls, multi-hop completion rate.                             |
| Math      | Exact answer accuracy, arithmetic error rate, substitution-check rate, constraint-check rate, MATH-500 accuracy.                      |

### 11.3 Success criteria

- E3 improves over E1 and E2 on answer accuracy.
- E3 improves verification-present and verification-valid rates.

- E3 reduces premature final answers.
- Search-QA gains are strongest on multi-hop questions.
- Math gains are strongest on problems where verification catches arithmetic or setup errors.
- Branch count and tool calls do not explode.

## 12. Implementation order

| Step | Task                                                   | Exit criterion                                                             |
|------|--------------------------------------------------------|----------------------------------------------------------------------------|
| 1    | Build bracket parser and format checker.               | Can parse <plan>, <think>, <tool>, <reflect>, <final> from generated text. |
| 2    | Build evaluators for Search-QA and Math.               | Can score EM/F1 or exact answer and detect verification presence.          |
| 3    | Generate 100 HotpotQA and 100 GSM8K original rollouts. | Manual inspection shows reasonable format adherence.                       |
| 4    | Generate teacher corrections for the same examples.    | Corrected rollouts are valid and mostly correct.                           |
| 5    | Build first 100-200 DPO pairs.                         | DPO dataset loads and trains without errors.                               |
| 6    | Run 500+500 dry run.                                   | Compare E1/E2/E3 on small validation.                                      |
| 7    | Scale to real V1 dataset.                              | Run full evaluation table.                                                 |

## 13. Hyperparameters

| Parameter                      | Starting value                 |
|--------------------------------|--------------------------------|
| Student                        | Qwen3-4B-Instruct              |
| Training method                | LoRA or QLoRA                  |
| Reference model                | Frozen copy of initial student |
| Learning rate                  | 5e-6 to 1e-5                   |
| DPO beta                       | 0.1                            |
| Epochs                         | 1 initially                    |
| Rollout temperature            | 0.7                            |
| Teacher correction temperature | 0.2                            |
| Max prompt length              | 2048 or 4096                   |
| Max completion length          | 1536 or 2048                   |
| Format SFT weight              | 0.1                            |

## 14. Later extensions, not V1

- Student-regenerated corrected rollouts instead of teacher-written corrected rollouts.
- Minimal feedback levels and reveal-cost preferences.
- Utility ranking: correctness minus tool cost, branch cost, and reveal cost.

- Memory tools: `memory.write`, `memory.read`, `memory.update`, `memory.timeline`.
- Trainable feedbacker or teacher using a Bi-NAC-style objective.
- GRPO/RL after DPO stabilizes behavior.
- OpenHands/Qwen Code/SWE-agent integration for coding-agent tasks.

#### Why these are later

These extensions are useful, but they would make the first result hard to debug. V1 should first establish whether the core preference-learning signal works.

## 15. References and source notes

These are the main papers/datasets/tools motivating the design. The PDF avoids relying on these as implementation dependencies; they are included to ground the benchmark/model choices and loss design.

- Rafailov et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290.
- Yang et al. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. arXiv:1809.09600.
- Kwiatkowski et al. Natural Questions: A Benchmark for Question Answering Research. TACL 2019.
- Cobbe et al. Training Verifiers to Solve Math Word Problems / GSM8K. arXiv:2110.14168.
- Hendrycks et al. Measuring Mathematical Problem Solving With the MATH Dataset. arXiv:2103.03874.
- Ho et al. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps / 2WikiMultiHopQA. COLING 2020.
- Trivedi et al. MuSiQue: Multi-hop Questions via Single-hop Question Composition. TACL 2022.
- Qwen Team. Qwen3 Technical Report. arXiv:2505.09388.
- Hugging Face TRL documentation: DPOTrainer.

## Final V1 summary

1. Student: Qwen3-4B-Instruct.
2. Teacher: frozen stronger model.
3. Domains: HotpotQA + GSM8K first.
4. Format: `<plan>`, `<think>`, `<tool>`, `<reflect>`, `<final>`.
5. Reflection must verify before final answer.
6. Teacher gives textual feedback and corrected rollout.
7. DPO trains corrected rollout > original rollout.
8. Add 0.1 FormatSFT for stable structure.
9. Evaluate accuracy, verification, branch count, and tool calls.
10. Only after V1 works, add memory, teacher training, RL, and OpenHands/SWE.